

```

import React, { useState, useEffect, useRef, useCallback } from "react";
import { LucideIcon, X } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🌀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// ⓘ LOCAL TYPE ISOLATION GATEWAY
export interface HintActionConfig {
  icon: LucideIcon;
  onClick: (e: React.MouseEvent<HTMLButtonElement>) => void;
  tooltipText?: string;
}

export type HintPosition = "top" | "bottom" | "left" | "right";

export interface HintBoxProps {
  content: React.ReactNode;
  children: React.ReactNode;
  position?: HintPosition;
  leadingIcon?: LucideIcon;
  actions?: HintActionConfig[];
  className?: string;
  enterDelay?: number;
  leaveDelay?: number;
}

interface HintBoxState {
  isOpen: boolean;
  isHoveringTrigger: boolean;
  isHoveringContent: boolean;
}

interface PositionStyles {
  top?: string;
  left?: string;
  right?: string;
  bottom?: string;
  transform?: string;
}

const DEFAULT_ENTER_DELAY = 200;
const DEFAULT_LEAVE_DELAY = 150;
const MAX_ACTIONS = 2;

function getPositionStyles(
  position: HintPosition,

```

```

triggerRect: DOMRect | null,
contentRect: DOMRect | null,
viewportPadding = 8,
): PositionStyles {
  if (!triggerRect || !contentRect) {
    return {};
  }

  const viewportWidth = window.innerWidth;
  const viewportHeight = window.innerHeight;
  const gap = 8;

  let top = 0;
  let left = 0;
  let transform = "";

  switch (position) {
    case "top": {
      top = triggerRect.top - contentRect.height - gap;
      left = triggerRect.left + (triggerRect.width - contentRect.width) / 2;

      // Viewport boundary checks
      if (top < viewportPadding) {
        top = triggerRect.bottom + gap;
      }
      if (left < viewportPadding) {
        left = viewportPadding;
      }
      if (left + contentRect.width > viewportWidth - viewportPadding) {
        left = viewportWidth - contentRect.width - viewportPadding;
      }
      transform = "translateX(-50%) translateY(-100%)";
      break;
    }
    case "bottom": {
      top = triggerRect.bottom + gap;
      left = triggerRect.left + (triggerRect.width - contentRect.width) / 2;

      if (top + contentRect.height > viewportHeight - viewportPadding) {
        top = triggerRect.top - contentRect.height - gap;
      }
      if (left < viewportPadding) {
        left = viewportPadding;
      }
      if (left + contentRect.width > viewportWidth - viewportPadding) {
        left = viewportWidth - contentRect.width - viewportPadding;
      }
      transform = "translateX(-50%)";
      break;
    }
  }
}

```

```

case "left": {
  top = triggerRect.top + (triggerRect.height - contentRect.height) / 2;
  left = triggerRect.left - contentRect.width - gap;

  if (left < viewportPadding) {
    left = triggerRect.right + gap;
  }
  if (top < viewportPadding) {
    top = viewportPadding;
  }
  if (top + contentRect.height > viewportHeight - viewportPadding) {
    top = viewportHeight - contentRect.height - viewportPadding;
  }
  transform = "translateY(-50%) translateX(-100%)";
  break;
}
case "right": {
  top = triggerRect.top + (triggerRect.height - contentRect.height) / 2;
  left = triggerRect.right + gap;

  if (left + contentRect.width > viewportWidth - viewportPadding) {
    left = triggerRect.left - contentRect.width - gap;
  }
  if (top < viewportPadding) {
    top = viewportPadding;
  }
  if (top + contentRect.height > viewportHeight - viewportPadding) {
    top = viewportHeight - contentRect.height - viewportPadding;
  }
  transform = "translateY(-50%)";
  break;
}
}

return { top: `${top}px`, left: `${left}px`, transform };
}

```

```

function getArrowStyles(position: HintPosition): React.CSSProperties {
  const baseStyles: React.CSSProperties = {
    width: 0,
    height: 0,
    borderLeft: "6px solid transparent",
    borderRight: "6px solid transparent",
    position: "absolute",
    pointerEvents: "none",
  };
};

```

```

switch (position) {
  case "top":
    return {

```

```

        ...baseStyles,
        borderTop: "6px solid",
        bottom: "-6px",
        left: "50%",
        transform: "translateX(-50%) rotate(180deg)",
    };
    case "bottom":
    return {
        ...baseStyles,
        borderBottom: "6px solid",
        top: "-6px",
        left: "50%",
        transform: "translateX(-50%)",
    };
    case "left":
    return {
        ...baseStyles,
        borderLeft: "6px solid",
        right: "-6px",
        top: "50%",
        transform: "translateY(-50%) rotate(90deg)",
    };
    case "right":
    return {
        ...baseStyles,
        borderRight: "6px solid",
        left: "-6px",
        top: "50%",
        transform: "translateY(-50%) rotate(-90deg)",
    };
}
}

```

```

export function HintBox({
    content,
    children,
    position = "top",
    leadingIcon: LeadingIcon,
    actions = [],
    className,
    enterDelay = DEFAULT_ENTER_DELAY,
    leaveDelay = DEFAULT_LEAVE_DELAY,
}: HintBoxProps) {
    const [state, setState] = useState<HintBoxState>({
        isOpen: false,
        isHoveringTrigger: false,
        isHoveringContent: false,
    });

    const triggerRef = useRef<HTMLDivElement>(null);

```

```

const contentRef = useRef<HTMLDivElement>(null);
const portalRef = useRef<HTMLDivElement>(null);
const enterTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
const leaveTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
const positionStylesRef = useRef<PositionStyles>({});

const updatePosition = useCallback(() => {
  if (triggerRef.current && contentRef.current) {
    const trigger = triggerRef.current.getBoundingClientRect();
    const content = contentRef.current.getBoundingClientRect();
    positionStylesRef.current = getPositionStyles(position, trigger, content);
  }
}, [position]);

useEffect(() => {
  updatePosition();
  window.addEventListener("resize", updatePosition);
  window.addEventListener("scroll", updatePosition, true);
  return () => {
    window.removeEventListener("resize", updatePosition);
    window.removeEventListener("scroll", updatePosition, true);
  };
}, [updatePosition]);

const clearTimeouts = useCallback(() => {
  if (enterTimeoutRef.current) {
    clearTimeout(enterTimeoutRef.current);
  }
  if (leaveTimeoutRef.current) {
    clearTimeout(leaveTimeoutRef.current);
  }
}, []);

const handleTriggerEnter = useCallback(() => {
  clearTimeouts();
  setState((prev) => ({ ...prev, isHoveringTrigger: true }));
  enterTimeoutRef.current = setTimeout(() => {
    setState((prev) => ({ ...prev, isOpen: true }));
  }, enterDelay);
}, [clearTimeouts, enterDelay]);

const handleTriggerLeave = useCallback(() => {
  clearTimeouts();
  setState((prev) => ({ ...prev, isHoveringTrigger: false }));
  leaveTimeoutRef.current = setTimeout(() => {
    setState((prev) => {
      if (!prev.isHoveringContent) {
        return { ...prev, isOpen: false };
      }
      return prev;
    });
  }, leaveDelay);
}, [clearTimeouts, leaveDelay]);

```

```

    });
    }, leaveDelay);
  }, [clearTimeouts, leaveDelay]);

const handleContentEnter = useCallback(() => {
  clearTimeouts();
  setState((prev) => ({ ...prev, isHoveringContent: true, isOpen: true }));
}, [clearTimeouts]);

const handleContentLeave = useCallback(() => {
  clearTimeouts();
  setState((prev) => ({ ...prev, isHoveringContent: false }));
  leaveTimeoutRef.current = setTimeout(() => {
    setState((prev) => {
      if (!prev.isHoveringTrigger) {
        return { ...prev, isOpen: false };
      }
      return prev;
    });
  }, leaveDelay);
}, [clearTimeouts, leaveDelay]);

const handleActionClick = useCallback(
  (action: HintActionConfig, e: React.MouseEvent<HTMLButtonElement>) => {
    action.onClick(e);
    if (action.icon === X || action.tooltipText === "Dismiss") {
      setState((prev) => ({ ...prev, isOpen: false }));
    }
  },
  [],
);

const handleKeyDown = useCallback((e: React.KeyboardEvent) => {
  if (e.key === "Escape") {
    setState((prev) => ({ ...prev, isOpen: false }));
  }
}, []);

const triggerElement = React.Children.only(children) as React.ReactElement;
const enhancedTrigger = (
  <div
    ref={triggerRef}
    className="relative inline-block"
    onMouseEnter={handleTriggerEnter}
    onMouseLeave={handleTriggerLeave}
    onFocus={handleTriggerEnter}
    onBlur={handleTriggerLeave}
    onKeyDown={handleKeyDown}
    aria-describedby={state.isOpen ? "hint-box-content" : undefined}
  >

```

```

    {triggerElement}
  </div>
);

const limitedActions = actions.slice(0, MAX_ACTIONS);

const contentElement = (
  <div
    ref={contentRef}
    id="hint-box-content"
    role="tooltip"
    style={positionStylesRef.current as React.CSSProperties}
    className={cn(
      "fixed z-50 pointer-events-auto",
      "bg-card backdrop-blur-[var(--backdrop-blur)]",
      "border border-[color-mix(in_oklch_var(--border)_50%_transparent)]",
      "rounded-surface",
      "shadow-lg",
      "p-3",
      "max-w-xs",
      "text-sm text-card-foreground",
      "font-sans",
      "data-[state=open]:animate-in data-[state=closed]:animate-out",
      "data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
      "data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95",
      "data-[side=bottom]:slide-in-from-top-2",
data-[side=top]:slide-in-from-bottom-2",
      "data-[side=left]:slide-in-from-right-2",
data-[side=right]:slide-in-from-left-2",
      className,
    )}
    data-state={state.isOpen ? "open" : "closed"}
    data-side={position}
    onMouseEnter={handleContentEnter}
    onMouseLeave={handleContentLeave}
  >
    <div className="flex items-start gap-2.5">
      {LeadingIcon && (
        <div
          className={cn(
            "flex-shrink-0 flex items-center justify-center",
            "w-5 h-5",
            "text-muted-foreground/80",
          )}
          aria-hidden="true"
        >
          <LeadingIcon className="w-4 h-4" />
        </div>
      )}
      <div className="flex-1 min-w-0">{content}</div>

```

```

    {limitedActions.length > 0 && (
      <div className="flex items-center gap-1.5 flex-shrink-0">
        {limitedActions.map((action, index) => (
          <button
            key={index}
            type="button"
            onClick={(e) => handleActionClick(action, e)}
            className={cn(
              "relative flex items-center justify-center",
              "w-7 h-7 rounded-md",
              "text-muted-foreground/60 hover:text-foreground",
              "bg-transparent hover:bg-accent",
              "transition-all duration-150 ease-out",
              "active:scale-95",
              "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2",
              "disabled:opacity-50 disabled:pointer-events-none",
            )}
            aria-label={action.tooltipText || "Action"}
            disabled={!action.onClick}
          >
            <action.icon className="w-3.5 h-3.5" aria-hidden="true" />
          </button>
        ))}
      </div>
    )}
  </div>
  <div
    style={getArrowStyles(position)}
    className="pointer-events-none"
    aria-hidden="true"
  />
</div>
);

return (
  <>
    {enhancedTrigger}
    {state.isOpen && (
      <div
        ref={portalRef}
        className="fixed inset-0 z-40 pointer-events-none"
        aria-hidden="true"
      >
        {contentElement}
      </div>
    )}
  </>
);
}

```



```
export default HintBox;
```